Certifying No Effect and Identifying the Win Condition Under Per-Action Scoring in ARC-AGI-3

Julio Cesar Silveira de Souza

Abstract

ARC-AGI-3 evaluates goal-setting, identifying desirable states without instructions, yet each game fixes its win condition, and the score measures how economically it is reached. By Ackoff and Emery's taxonomy, competing agents are at most multi-goal-seeking. Our working definition of a general intelligence, broader than the AGI the benchmark defines, is a purposeful system, selecting goals as well as means, and we study the step within reach, identifying a game's goal without the actions the score charges. Our agent adds to an open-weight large language model (LLM) harness a graph-based explorer issuing revocable no-effect certificates, detecting hidden state when one fails or a safe action kills, and ranking goal hypotheses that only order actions. With the LLM still the default policy, the layer is a measurement instrument whose presence we vary. With the layer playing, our first scored submission reached 0.75 on another base, against 26.74 for the unmodified dfranzen base, unpaired but compatible with action cost. Advising through prompt notes and list pruning, the layer had no measurable effect in two three-round ablations five days apart, with the threshold fixed after a one-round pilot. In last-resort mode, one post-deadline hidden-set draw gave 28.41 against 26.74, inside the base's spread. With default goal families, 50 of 66 local wins, unnamed by the identifier, had no implemented predicate true in the winning frame. We report a measured gap in goal-setting, one of the benchmark's four components, and a direction beyond this competition.

1 Introduction

ARC-AGI-3 is the third benchmark in the ARC-AGI series and the first fully interactive one, whereas ARC-AGI-1 (2019) and ARC-AGI-2 (2025) consisted of static grid tasks (ARC Prize Foundation, 2026, p. 1; Kamradt, 2026a). ARC-AGI-3 places an agent within a novel, turn-based game whose rules, mechanics, and win conditions are never stated, and the agent inspects a 64×64 grid and selects from a limited discrete action set. It is evaluated by Relative Human Action Efficiency (RHAE), which takes `min(1.15, (h/a)^2)` per level against an upper-median first-run human baseline, weighted by level index. The paper that introduces the benchmark names four components of agentic intelligence that it evaluates: exploration, modeling, goal-setting, and planning and execution. At release, frontier systems scored below 1%, and humans solved every environment (ARC Prize Foundation, 2026, pp. 1, 5, 6, 12, 19). By September, on the semi-private set, a frontier model scored 62.7% under the foundation's standard harness at maximum reasoning effort and 99.9% at high effort in a harness that preserves its provider's reasoning state, where, at maximum effort, it used fewer actions than the median human on 96% of levels (Kamradt, 2026b). This paper, instead, works in the competition's setting, offline, with no model API and a nine-hour limit, where our agent runs an open-weight model on a single GPU (Section 3.1).

A recent line of work builds executable world models and verifies them against interaction history (Rodionov, 2026, p. 1; Courtis et al., 2026, p. 2; Skoutnev et al., 2026, p. 3; Lehmann et al., 2026, p.

1). Two of them, described in Section 2, measure their bottleneck, the goal. In Twin, dynamics are learnable, whereas the goal predicate is not yet stated reliably (Skoutnev et al., 2026, p. 28). In Tycho, the policy with the most exact transition models scores below another policy in the same comparison, 83.07 against 88.49 RHAE (Lehmann et al., 2026, pp. 1, 24). In these systems, how the world moves is increasingly certified, but what would count as winning is not yet established reliably. Nor, as we frame the problem, is it established which actions can be safely disregarded while finding out. A seventh-placed competition team measured the same gap from the other side: given the written rules of each public game and a hint per level, its model "cleared 70% more levels on the hardest games and scored 85.7 instead of 64.4 on the 25 public games," so that "most of the gap is in working that out" (Sestak et al., 2026). The frontier model mentioned above followed a related path in its notes, building "compact symbolic world models" (Kamradt, 2026b). A discovery gap persists without the visual channel, as in DiG-bench (Battleday et al., 2026, p. 6), 70 text games whose rules and win conditions are hidden, where Gemini 3.1 Pro won 18 games, and 69 when given a brief description of the dynamics and the win condition. Where a comparison with the same model was available, the agentic harnesses tested there, PRO-LONG (Fox et al., 2026) and Prime Agent among them, performed no better than the basic one on the two hardest tiers.

These games are built and play-tested by humans, and each enters the benchmark only if at least two of the ten people who first attempt it solve it in full (ARC Prize Foundation, 2026, pp. 7, 16). They draw only on Core Knowledge priors (p. 8), the systems for representing objects, agents, number, and space that Spelke and Kinzler (2007, p. 89) describe. Tolman (1948, pp. 198–199) observed that, in discrimination experiments, human subjects arrive with their instructions, whereas his rats "do not know what is wanted of them," so that most of their learning seems to consist in discovering the instructions. ARC-AGI-3 withholds the instructions from every player, and the agent "must autonomously infer" the win condition (ARC Prize Foundation, 2026, p. 5).

In the absence of instructions, the most informative move is frequently a probe, an action selected to differentiate between two competing hypotheses about one of the game's rules (for instance, whether touching an object moves it or removes it), even if it deviates from the apparent objective. Assessed by progress toward that objective, such a probe appears to be a regression. Lehman and Stanley (2011, pp. 190, 193) refer to problems of this type, whose progress signal points away from the solution, as deceptive. We conjecture that a policy driven by a large language model (LLM) under-proposes such probes, at least an open-weight model of the size that the competition allows. The system prompt of the dfranzen base we build on (Franzen, 2026a; Section 3.1) already tells the model, "If confidence is low, program a discriminating probe." Even so, the model samples what its training makes probable, and it reasons within rules literalized from a few transitions. In our system, the potentially deceptive signal is explicit: the progress scalar of each goal family (Section 3.3). This is why no hypothesis may forbid an action.

Forbidding no action keeps the search open, but it leaves a prior question untouched, namely who sets the goal that the search pursues, and the benchmark answers part of it by design. The ARC Prize Foundation defines AGI as the "ability to acquire any skill a human can, as efficiently as a human can" (ARC Prize Foundation, 2026, p. 4). Goal-setting, one of the four components, is "the ability to identify interesting or desirable future states without explicit instructions," and the agent must determine "what to target" from its "own intrinsic drive and environmental cues" (p. 5). The foundation defines intelligence as efficiency across these components (p. 5) and reads progress toward AGI only on unseen environments, without task-specific harnesses (p. 15). Chollet, who designed the benchmark and co-founded the foundation (pp. 1–2), defines intelligence as skill-acquisition efficiency and built the original ARC to measure "the kind of general intelligence" he defined (Chollet, 2019,

pp. 27, 46). The literature on intelligence and on AGI answers that question in many other ways, and we do not survey it here. We delimit the contrast to the definitions that the benchmark adopts, which belong to one lineage, set against a reading from systems thinking that inspires our approach. That reading is the taxonomy of Ackoff and Emery (1972/2017, pp. 30–31), ordered by who chooses the goal. A goal-seeking system responds to events in different ways, but all its responses serve to produce one particular outcome, its goal. A multi-goal-seeking system is goal-seeking in several environments and seeks different goals in at least two of them, and the environment rather than the system determines which, as in a computer that plays several games and is told which one to play by "an external source." A purposeful system "can change its goals in constant environmental conditions," selecting goals as well as means.

As we read this lineage, the skills to be acquired and the goals they serve are fixed outside the system being measured, even when it is not told them. Chollet states the scope explicitly: ARC "only assesses a general form of fluid intelligence, with a focus on reasoning and abstraction," in a format deliberately close to IQ tests (2019, p. 50). He also sets aside the Turing test, which delegates the judgment to human judges, as a "red herring" for the field (p. 3). Two readings from outside this lineage suffice for our contrast. Ackoff and Emery note the uncertainty "about what is actually being measured by so-called IQ tests" and relate intelligence to the rate at which a subject learns. They distinguish the intelligence measured by IQ tests from a creative intelligence that incorporates aspects of environmental and self-awareness, which they find closest to Chein's definition, in which relevance is set by "the immediate and presumptive future purposes of the actor." They also point to "the difficulty of trying to represent intelligence by a single number" (1972/2017, pp. 52–53). Maturana and Guiloff (1980), from biology, argue that intelligence is not directly observable and "cannot be measured," an IQ test estimating at most "a subdomain of the domain of consensuality between the observer and the subject" (p. 135). They further argue that a goal is not operative in a system's own dynamics and belongs instead to the description made by an observer (p. 140).

Read together, the two vocabularies speak of the same object. The outcome that a system's responses serve to produce in Ackoff and Emery's account is the desirable future state that the foundation's goal-setting targets. Setting a goal, in the foundation's sense, is identifying the state to target without being told, and seeking it, in Ackoff and Emery's sense, is acting to produce it. Their taxonomy asks in addition whether the system or its environment did the determining, and on that question neither reading places it in the environment. Ackoff and Emery tie intelligence to the actor's purposes, and Maturana and Guiloff place the goal in an observer's description. In ARC-AGI-3, the environment is the game, and its hidden rules fix the win condition that ends each level. The score is not one of those rules. The foundation computes it from outside, comparing the agent's actions per level with those of human players on the same games, which is RHAE (pp. 11, 13), so that it measures how economically each win condition is reached and not which goal was chosen. The reference is an upper-median best first-run human playthrough rather than the most efficient play, which the foundation also estimates as an optimal playthrough once a game's mechanics and goals are understood (ARC Prize Foundation, 2026, p. 19). That reference is drawn from about 500 members of the public "not selected for puzzle-solving experience or ability" (Kamradt, 2026b). Within the environment, goal-setting therefore amounts to identifying a goal that the game's rules already determine, and the agents that compete, ours included, are at most multi-goal-seeking. Figure 1 sets this reading beside the behavioral classification of systems that Gharajedaghi (2011, p. 36) draws from Ackoff, in which a goal-seeking system has a choice of means but not of ends. Such a system, if it has memory, "can learn to pursue its goal more efficiently over time" (p. 37), which is what a per-action score rewards, whereas a purposeful system "can change its ends under constant conditions" (p. 37).

Behavior Process	Means Structure	End Function	Reading in Section 1
Passive tools	No choice; one structure in all environments	No choice; one function in all environments	
Reactive state-maintaining	No choice; variable but determined	No choice; one function in all different environments	
Responsive goal-seeking	Choice of means; variable and chosen	No choice of ends; variable but determined	The agents of ARC-AGI-3, ours included. Each game's rules fix the end, its win condition; the score charges the means, actions per level against a human reference. Across games, at most multi-goal-seeking: the game, not the agent, sets which goal (Ackoff & Emery).
Active purposeful	Choice of means; variable and chosen	Choice of ends; variable and chosen	The working definition of a general intelligence adopted here: a purposeful system that selects its goals as well as its means. Not tested.

Left three columns: Gharajedaghi (2011, p. 36, Table 2.1), after Ackoff.

Figure 1: Behavioral classification of systems in Gharajedaghi (2011, p. 36, Table 2.1), after Ackoff, with the reading of Section 1 beside it. The agents of ARC-AGI-3 choose their means and not their ends, and the score charges the means; the working definition adopted here asks for the choice of ends as well.

The instruction that the games withhold reappears upstream. In Ackoff and Emery's example, an external source tells the computer which game to play. Here, that source is the developer, who chooses the agent's priors, its tools, and, in our case, the goal families among which it searches (Section 3.3). The authors of the Duck harness, on which both of our bases build, report the same from their side: to make the agent perform well, they "had to encode a lot of specific knowledge into the prompts," including "steering its goal setting," for instance keeping it from treating the energy bar as the objective (Bessis et al., 2026b). Roli, Jaeger, and Kauffman (2022), for whom no system in an algorithmic frame can "choose and define their own goals" (p. 8), place the choice in the same hands. The designers set the criteria of learning, and even task-agnostic functions are "the result of an imposed external choice" (p. 9). The foundation draws the same line from its side, since ARC-AGI-3 is not meant to measure "the amount of human intelligence that went into designing" a system built for it (ARC Prize Foundation, 2026, p. 15). What the prize seeks from that intelligence is, as we read it, the ideas behind the designs, since participants must open source their solutions to receive prize money (p. 21). How much of the identification a developer can supply is visible in the foundation's measurements. In one public environment, the same frontier model scored 0.0% without a harness and 97.1% with a handcrafted one, and 0.0% under both in another, which the foundation reads as evidence that such harnesses do not carry over to unseen environments (p. 15). Its reading of AGI, in turn, is tied to a minimal, provider-neutral harness: "a future AGI should be able to solve ARC-AGI-3 under these conditions" (Kamradt, 2026b).

Both vocabularies measure how well ends are pursued, as efficiency across the benchmark's components (ARC Prize Foundation, 2026, p. 5) or as the rate at which a subject learns, a subject being, for Ackoff and Emery, a purposeful individual or system (1972/2017, pp. 34, 52–53). What separates them is where the ends come from. For this paper we therefore adopt a working definition of our own,

narrower than a survey of the literature would support. A general intelligence, in this sense, would be a purposeful system, one that selects its goals as well as its means, whose intelligence includes the creative kind that Ackoff and Emery associate with environmental and self-awareness and find closest to Chein's definition, relative to "the immediate and presumptive future purposes of the actor" itself (1972/2017, p. 52). We conjecture that such a capacity would emerge in the system rather than be designed into it. By this definition, which is not the one the benchmark adopts, choosing goals is the condition that still separates the agents studied here from a general one. We do not test awareness, and we make no claim about consciousness or sentience. The foundation's account of the environments points the same way, since it describes them as having "deterministic, closed-ended mechanics and goals" and does not read saturating the benchmark as "proof of achieving AGI." It also names open-ended innovation among the questions for the next generation of benchmarks (Kamradt, 2026b).

If choosing goals lies outside what the benchmark asks, the step an agent can take within it, and the one ours is built for, is the identification of the goal each game fixes, without spending the actions that the score charges. That step joins the two open questions of the second paragraph, and we address both from the opposite end of the literature reviewed there, since we do not learn a world model. Instead, the agent holds concurrent hypotheses about the goal, none of which may forbid an action, and records which actions have shown no effect, so that both claims can be checked against play. Because the agent's default policy is the language model rather than our layer, the question is whether these claims are correct and whether they reach the model's choices. We therefore treat the component that makes them as a measurement instrument and vary how it is present in the agent, whether in the playing mode, the advising mode, or the last-resort mode (Section 5). Concretely, our agent is a graph-based explorer coupled to the harness of an LLM agent. The explorer runs breadth-first search (BFS) to the frontier of untested state–action pairs, in the line of Rudakov et al. (2025, p. 1). The harness was first a fork of the Duck harness (Bessis et al., 2026a) and then the dfranzen notebook (Franzen, 2026a), both described in Section 3.1. Our contributions are:

1. **A verifiable no-effect certificate** (Section 3.2): a logged, revocable claim that an action changes nothing at a state, masked from the model's list in the configurations that prune (Section 5). Recording and revoking such claims has antecedents, the dfranzen base's guard among them (Section 2).
2. **Hidden-state detection by certificate failure or unexpected death** (Section 3.2), with per-game counters (per trigger in the later versions).
3. **A measured win-condition identification component** (H5, Section 3.3): concurrent goal hypotheses whose leader only orders candidate actions, and whose named failure is a first-class result. With the default families, 50 of 66 local wins have no true hypothesis instance in the winning frame, and an eight-arm attribution bench finds no measurable score effect of any family, of a broad proximity family, or of randomized ordering.
4. **An adversarial verification protocol** (Section 4; pre-registered invariants, independent blue/red rounds, mutation testing, hash-frozen artifacts) and the defect taxonomy it produced in the Duck line, where most defects lay between what the agent asserts and what the environment permits.

2 Prior art

Graph exploration without a model. Rudakov et al. (2025, pp. 1, 3–4) segment frames into single-color connected components, hash the states, and then search the frontier of untested state–action pairs. Their training-free explorer solves a median of 30 of 52 preview levels and outperforms the best LLM-based entry on the preview leaderboard, whose official score they report without rerunning it. The explorer's cost grows with the state space. Our explorer inherits this skeleton, and the principle is older. Go-Explore remembers visited states and first returns to a promising one, without exploring, before exploring from it. It is a graph search over conflated states whose edges must be discovered (Ecoffet et al., 2021, pp. 1, 3, 20). The benchmark's validator builds such a graph, with hash-based nodes merged across trajectories and invalid actions recorded as transitions that do not change the state. Its nodes also carry the environment's hidden state (ARC Prize Foundation, 2026, p. 9), whereas our masked-frame states do not, which is what our hidden-state triggers watch for.

Executable world models. Rodionov (2026, pp. 1, 4) has a coding agent maintain a Python world model. The model is verified against observations and refactored toward simpler abstractions, and the plan executor stops at the first divergence. OPINE-World (Courtis et al., 2026, pp. 2, 18, 27) couples a natural-language proto-model, a Bayesian belief state over type-action effects, and a replay-verified program model. The belief state tracks mixed effects as uncertainty and tests ontology refinements, and removing the Bayesian layer costs 20.24 RHAE points (78.40 → 58.16). Twin (Skoutnev et al., 2026, pp. 1, 3, 22, 28) refuses any scored action until its executable twin reproduces the *entire* interaction log, and it hypothesizes the goal before any reward. The first goal hypothesis is correct on 87.2% of cleared levels, but goal precision over all claims is 0.214 against 79.4% online dynamics accuracy (all predictions pooled). Tycho (Lehmann et al., 2026, pp. 1, 3, 24) formalizes games as rendered Moore machines and compares four model-maintenance policies in one public-set run per policy under matched inference budgets. Automatic repair attains the highest transition match (88.1%) and does not score best, so that transition accuracy and score come apart in that comparison.

Explore, then solve. Liew (2026, pp. 1–3) gates an EXPLORE/VERIFY/PLAN agent on a proxy for belief entropy. The study finds every one of the 25 public games reachable by non-intelligent strategies, so that the public set "cannot discriminate intelligent exploration from trivial heuristics" (p. 1). The caution applies to our local bench (Section 5), which is built on the same games. Tsividis et al. (2021, pp. 3–5) learn win and loss conditions as part of a Bayesian program model (EMPA), with theory-based curiosity generating exploratory goals. Section 3.3 follows them in treating the goal as an object to be learned.

No-op knowledge in harnesses. In a Duck fork, the `agentfix` module of Le Grand (2026) has switches that measure the effect of every executed action outside the HUD (display) band, annotate repetitions of a proven (state, action) no-op (`repeat_of_known_no_op`), and refuse such repetitions. The refusal is bounded and lets every third attempt re-verify. All three switches are off in the published version's defaults, and the refusal is off because, per its notes, refusals "misled the model" in an earlier version. In another Duck fork, the arc-3 lab reports no-impact detection, the stopping of an exploratory action whose only board change is the game's HUD/moves band, as the one clear win of its ablations (sonpham-org, 2026). The mean of two passes, excluding the game ft09, gave 1.62 against 1.05 and 21 against 15 levels at an equal action budget. The lab warns that a single run cannot resolve a harness change. In a third fork, on the dfranzen base, an observation-based prefix verifier, tested apart from the language model on the public games, did no better than checking

after every action, and it was shipped switched off (Shamim, 2026, pp. 1, 10). The same entry's learned explorer, which penalizes actions with a no-op history, made fewer no-ops than a random policy (31.05 against 47.69 per run) and did not score higher (pp. 9, 11). Predicting which actions change the frame also drove the winning entry of the 2025 preview, StochasticGoose, a convolutional network trained with reinforcement learning (ARC Prize Foundation, 2026, p. 20). The dfranzen base itself ships a guard, off by default (Franzen, 2026a). It records border-masked (state, action) no-ops per level, refuses a repeat unless the model at once re-issues it, and drops an entry that later evidence contradicts. Its author reports that remembered no-op guards, among others, were explored "without sufficiently clear benefit to include in this configuration" (Franzen, 2026b). Our certificate keeps that record and that revocation, and we add and measure the response to a contradiction, by which the level is marked hidden-state and its other certificates are voided. The list mask and the per-trigger counters follow from that.

Goal induction from level transitions. The Misfit agent (McCree, 2026) induces ranked goal predicates from observed transitions (support requires a level change) in three families: `removed_all_of_class`, `agent_reached_class`, and `count_of_class_equals_N`. The predicates are scored by a Laplace-smoothed posterior in which a level advancing without the predicate counts as a contradiction. The top-ranked hypothesis at a win is persisted as a tag, and no action is taken from it. Relative to that implementation, Section 3.3 uses the leading hypothesis to order actions and measures the result.

Same base, other levers. Concurrent work on the same dfranzen base (Lam Dang, 2026) kept the agent unchanged and worked on the inference engine. The changes were expert pruning, concurrency, faster loading, and then scheduler and cache fixes, and they raised the leaderboard score from about 26 to 33.68 and then to 37.75 (scored after the deadline). Every harness change it also submitted scored below the unchanged harness, including a guard against repeating known no-op actions ($+3.9 \pm 3.5$ on public games, -8.0 on the leaderboard, one leaderboard draw), and public validation did not track the leaderboard for agent changes. In its account, under a fixed wall-clock budget, the number of tokens the model can generate per game bounds what the agent can do. A layer like ours draws on that same budget. Another team on the same base, with the patch declared identical, reached 43.73 by serving and a scheduler that keeps 39 to 48 games active instead of 10, and it reports no ablation of its harness changes (Taguchi et al., 2026). After the competition, the sixth-placed participant reported the same split, in which serving optimization was "the BIGGEST thing to improve," and harness changes "died in the noise" (Zatravkin, 2026b). Public games were a weak predictor even for a serving setting, since Team Pandora's record public-game run (53.51) scored 26.90 on the hidden set, below a setting that had scored lower on the public games (27.38; Team Pandora, 2026).

3 Method

3.1 The explorer (K) inside the harness

The system couples K to the harness of an LLM agent through a single analyzer hook, and two bases were used, both with that hook. K and its certificates come from a single code lineage that is versioned and hash-frozen per submission. The first is a fork of the Duck harness (Bessis et al., 2026a), in which a local language model writes and executes Python in repeated short tool calls. It produced our first scored submission (leaderboard 0.75, Section 5), which predates H5, and was then retired. We moved to the dfranzen base after reading the competition's public notebooks, when the distance between our scores and the leading entries showed that the explorer

would not score competitively under the competition's conditions. The scoring public notebooks derived from that base, and running it unmodified gave the paired control for the modes that followed. The second is the dfranzen Milestone-2 notebook (Franzen, 2026a), itself a fork of the Duck harness and released under Apache-2.0 in its author's repository. We used it as published, not through a third party's copy, and it carries every later version. The Duck line ran Qwen3.6-27B in FP8 (vrfai/Qwen3.6-27B-FP8, loaded from the Kaggle dataset `driessmit1/vrfai-qwen3-6-27b-fp8-hf-snapshot`). The dfranzen line runs the base's model, Intel-Qwen3.8-Flash-Next-W4A16-AutoRound (Kaggle model `dfranzen/intel-qwen3.8-flash-next-w4a16-autoround`), served with a speculative-decoding drafter (`dfranzen/albucino-qwen3-8-flash-next-drafter`). We took each base's published model and serving as the reference and did not optimize either. In both lines, our changes to the base notebook are the analyzer hook, whose settings (triggers and caps) are the only adaptation to K, and, outside competition reruns, a fixed three-game sample and time cap for phase A (Section 5). Differences between configurations on the same base therefore come from K's presence and not from compute, the condition that Section 8 proposes for a dedicated track.

We replace the exploration policy with K, a graph-based explorer in which frames are hashed into states with display-band cells masked. K maintains the directed graph of visited states and tested actions and plans breadth-first to the nearest state with untested actions. It also learns an action budget per life (the actions since the last reset or level start) when repeated deaths fall at the same count. K carries two fixed goal heuristics inherited from its lineage, moving a mover toward a target (H1) and matching a working region to a static reference (H2), and they only order candidates. H1 changes target when it stalls at one, and the guidance switches off for the level when H2's mismatch stops falling or the level's graph is exhausted. Both were written from a reading of the 25 public games that also form the local bench, and H5 (Section 3.3) replaces their guidance with its leading instance. When a level's graph is exhausted, K forgets every tested outcome of that level except deaths and tests again, at most eight times per level, alternating with a level reset.

In the first configuration (**playing mode**), the language model proposes actions by default, and K observes and, where configured, prunes the action list (Section 3.2). On named triggers (model failure, stall, repeated no-op, time pressure, with the yield measured in one variant), K takes the turn, and its graph notes enter the model's prompt. A later configuration (**advising mode**) disabled those handovers, so that no trigger gave K a turn and a model failure returned to the harness as in the unmodified base. The final configuration restricts K further (**last-resort mode**, Section 5), where prompt and action list are, by construction, byte-identical to the unmodified base until the final 600 seconds of each game's cap. Only then, with a level still open, may K play at most 150 actions per level and 600 per game. The prompt never uses the word "certificate," and in last-resort mode the model sees no explorer text at all. Version v14.1 adds the cross-game memory of Section 3.4.

The submitted notebook is rerun by Kaggle on the hidden game set, with internet disabled, as in every accelerated Kaggle session (ARC Prize Foundation, n.d.), and our notebook takes no human input. The hidden run plays 110 games once in one pass (Team Pandora, 2026; Zatravkin, 2026b). Of those, 55 are scored on the public and 55 on the private leaderboard, as reported by one participant (Zatravkin, 2026b).

3.2 No-effect certificates and hidden state (H8)

A **certificate** is issued for (state, action) when a single action, outside any batch, is executed on a level not yet marked hidden-state, the game does not end, and the successor frame differs from the current frame at most inside the masked display band. Certified keys are removed from the valid

action list given to the language model (action masking by observed no-op), *unless* removal would empty the list. In that case, the list is kept whole, and the advisory text says so (invariant I10, Section 4). A certificate is **falsified** when a later execution contradicts it, and falsification **revokes** the certificate and voids the other certificates of that level. Certificates of a level are also revoked when K reopens its tested actions (Section 3.1), so that what the mask had discarded returns to the model's list, but deaths are never reopened. All certificates are revoked when K relearns the display mask and reindexes its states.

A level is detected as **hidden-state** by either of two named triggers. Trigger (i) is falsification as above, including a death after a no-op even without a prior certificate. Trigger (ii) is a death on an action that K had already considered safe at that state (no effect, a change, or progress) or at the learned life budget (`morte_oculta`). In a hidden-state level, every later no-op statement in the model's prompt is downgraded from "proven to change NOTHING" to "changed nothing here before, but this level has hidden state, so that is NOT a proof." No-op results inherited across a state merge are similarly downgraded until re-observed. The run's summary table counts, per game, the issuances, the falsifications, the revocations at reindex and at reopening, and the masked keys, and it lists the hidden-state levels (by trigger from v11 on). Table 1 reports them for v4 (not submitted) and for v5 (the scored submission), in phase A of the Duck line (Section 5). Figure 2 draws the certificate's path and where K acts in each mode.

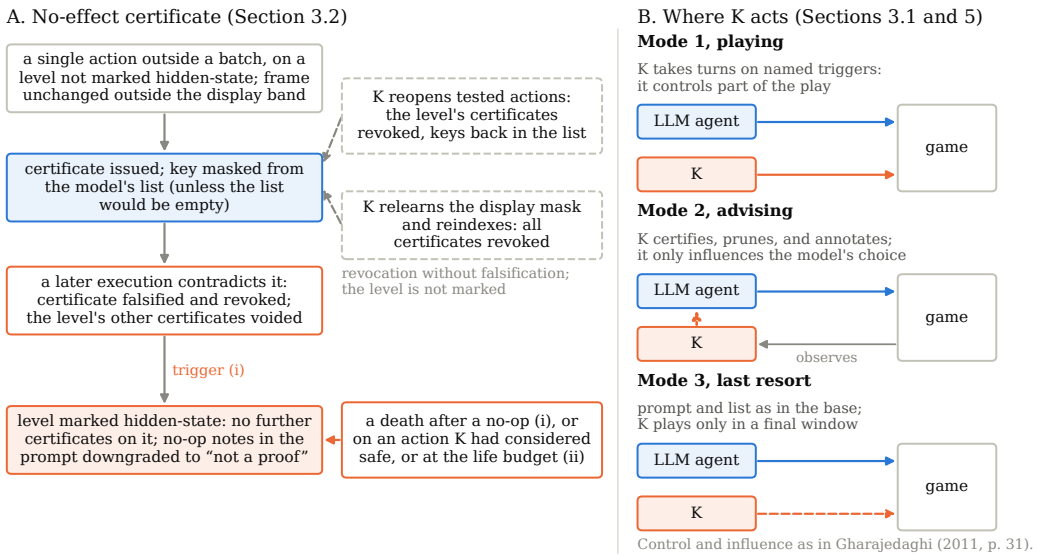


Figure 2: A, the no-effect certificate and the hidden-state mark (this section); no certificate is issued on a level once it is marked. B, where K acts in each mode of presence (Sections 3.1 and 5). In Gharajedaghi's terms, to control is to act in a way both necessary and sufficient for an outcome, and to influence is to act only as its coproducer (2011, p. 31). K controls part of the play in modes 1 and 3 and only influences the model in mode 2.

A certificate rests on what Bateson (1972) calls the assumption of repeatable context, "a necessary premise for any theory which defines 'learning' as change" (p. 292). Without it, he argues, "all would be zero learning" (p. 288), a specificity of response that "is not subject to correction" (p. 293). For Bateson the premise is the theorist's, but it "is not a mere tool of our description" and contains the hypothesis that the organism's experience is punctuated into contexts "which may be

equated or differentiated by the organism" (p. 292). Our agent turns that hypothesis into a test by assuming that the same masked hash, the counterpart of what Bateson calls a context marker (p. 290), marks the same context. Marking a level hidden-state withdraws that assumption for the level once its marker is shown to conflate two contexts, as in Bateson's account of experimental neurosis, where the context markers "become misleading" (p. 297).

Table 1: Certificate counters per game, phase A of the Duck line (three public games, 7,920 s cap per game, run concurrently; one draw). "Model on certified" counts model actions on a certified key; all revocations here are at reindex (none at reopening); masked keys are summed over prompts. Hidden-state levels are listed by index; the per-trigger counter was not printed in these runs, so the trigger is inferred (no certificate and no falsification ⇒ trigger (ii)). v4 predates the fix that made deaths on change or progress edges, or at the life budget, mark the level (Table 2, v5), so its hidden-state column is not comparable with v5's.

version	game	levels	model on certified	certified	falsified	revoked	masked keys	hidden-state levels
v4	ls20	1	25	142	0	119	167	—
v4	tu93	3	0	8	0	3	20	—
v4	vc33	2	27	233	2	5	0	1
v5	ls20	1	35	50	1	43	217	1
v5	tu93	3	0	0	0	0	0	3 (trigger (ii))
v5	vc33	1	20	84	5	0	0	0, 1

3.3 Win-condition identification (H5)

The component rests on the premise of this work, that an agent should arrive at a game with abstract techniques for discovering what is asked, rather than explore exhaustively and abstract afterward. Section 5 measures the exploration-first lineage that this premise argues against. On top of K, we add a goal-induction component (H5), closed into K's action loop, in the spirit of EMPA's learned termination conditions and Misfit's predicate families. On the model's turns, H5 only observes. Five predicate families are defined: mover-onto-target (O1), class removal (O3), region-equals-reference (G1), objects inside a closed contour (G2), and mirror symmetry (G4). Instances are created from observed frames once their evidence appears, so that class removal, for instance, is created only after a class is seen to vanish, and mirror symmetry only at the start of a level. Instances are evaluated on the known states of K's graph, and each carries a progress scalar that must decrease (for O3, the remaining cells of the class).

Tolman (1948, pp. 201–202) credits Krech with the notion of "hypotheses" in rats. These are systematic runs of choices, such as all right-hand doors and then all dark doors, which Tolman reads as cognitive maps set up "in a tentative fashion to be tried out first one and then another until, if possible, one is found which works" (p. 202). Our hypotheses are also tried in succession, though ranked by evidence. Unlike Krech's, which Tolman takes to show the "self-initiated character" of learning (p. 203), ours are not the agent's own, since the families are presuppositions of the designer. Churchman (1971) notes that all artificial intelligence machines "implicitly assume a capability that they are supposed to learn, provided we sweep in the presuppositions their designers used to build them" (p. 131). Our agent, likewise, revises hypotheses only inside a space we authored.

The leading hypothesis **only orders** candidates, selecting within the current tier of untried candidates and re-ranking frontier states within three steps of the nearest one, and it never excludes an action. A leader is replaced when it stalls (40 actions without improving its best progress), when the level's graph is exhausted, or when it is **eliminated**. A hypothesis is eliminated when its exact predicate becomes true in a stable frame, without a level change, after every legal action type in that state

has been tried. The goal it named was then reached, and the game did not end. At a win, instances are checked on the winning frame, and the family of the leader (if the leader is true) or of the best-ranked true instance receives extra evidence on the next level. Only a true leader counts as an identification (Section 5). The submitted versions disable O1 by default, after the red team, under the protocol of Section 4, measured a score cost over eight seeds and a latency that grows quadratically on 64×64 grids. The bench of Section 5 re-enables it as one arm.

3.4 Cross-game memory (submitted, v14.1)

Version v14.1 lets H5 carry something between games, under one rule: transfer the idea, not the move. A process-wide store, locked because the games run in parallel threads, keeps two tallies from unambiguous wins only (exactly one true instance in the winning frame): wins per predicate family and wins per abstract schema, a family together with the roles of its arguments. A role has no color and no coordinate. Each color becomes its rarity rank among the non-background colors of the level's first frame (capped at 3) plus a flag for whether it moves under action, so the role is invariant to color permutation, up to ties in color area. In-game evidence always decides. The schema only breaks ties for an instance already coupled to action (its progress fell under an action), and the family tally breaks ties after that. The arm counts as exercised in a game only if a decision of K had its leader changed by the store (`k_cross_decisao` > 0), a criterion fixed before the run.

The design follows an old result. Harlow (1949, pp. 52–56) gave eight monkeys 344 discrimination problems in a row, and in each the monkey had to find which of two new objects covered food. On the first problems, one trial with a new pair left the monkeys barely above chance on the next. On the last ones, that single trial was enough to choose correctly almost every time (p. 56). He called what they had acquired a learning set, "learning how to learn a kind of problem" (p. 53). What carried over was not the answers, since he doubted the monkeys could choose much above chance on the first trial of a problem they had already solved (p. 59). Bateson reads the same pattern in Hull's experiments on rote learning, where subjects improved from session to session. He attributes the gain to "some sort of adaptation to the context," since recalling the specific syllables "would more probably be a hindrance" (1972, p. 295). He classes Harlow's learning sets as "a rather special case" of the same category, Learning II, a change in the set of alternatives from which the learner chooses (pp. 293, 295–296). Our memory aims at that kind of transfer, keeping which kind of goal won, never the moves that won it. Ackoff and Emery describe a narrower learning in their goal-seeking system with memory, an electronic maze-solving rat, an automaton that can be programmed to take a solution path on later trials in a familiar maze (1972/2017, pp. 30–31). The cross-game memory targets the harder case, a new game with a different goal, the equivalent of an unfamiliar maze. Since it was not exercised in phase A (Section 5), we claim no learning in either sense.

4 Adversarial verification protocol

The agent was developed under a protocol fixed in a pre-registration written before the first version (27 October) and applied to every version of the Duck line. A systematic review in a neighboring field, machine learning for fluid-related equations, found weak baselines in 79% of the articles that claimed to outperform a standard method (McGreivy & Hakim, 2024, p. 1). It traced the overoptimism to analysis and reporting decisions made after the results were known, and it recommends pre-registration (pp. 8–9). The pre-registration set invariants (I1–I10): no exception from our layer reaching the harness, per-game state, legal actions, time limits, an untouched rerun, no network, embedded sources matching their hashes, and a mask that never empties the action list. It also set

claims (C1–C4): that phase A runs without exception and closes a level, that the rerun scores above zero, that a score prediction is sealed before the run, and that the counters are non-zero. A blue team implemented each version and its test suite, and an independent red team attacked it. The red team was always a different model from the blue team's, in a new session, with the frozen artifact and its hash manifest as the target. It delivered a verdict, RELEASE, RELEASE WITH RESERVATIONS, or DO NOT RELEASE (our translation of the Portuguese labels), with a reproducible proof per finding. Corrections returned with regression tests and with mutants, deliberate re-introductions of each defect that are required to fail the new tests. Passing tests resemble what Churchman, after Singer, calls readings, and "if the replicated readings agree, we cannot infer that the system is working properly" (Churchman, 1971, p. 191). His rule is to refine the inquiry "until the system reaches a level of refinement of its readings where not all readings agree" (p. 193). Mutants are our check that the tests can disagree, and their numbering reached M85 by v14.1, whose eight new mutants were all killed in its red round. No version with an open blocking finding was ever submitted, and the red round of v7 itself was superseded by that of v8.

Across the Duck-line versions (v1–v7), most of the defects that the red teams found share one character (Table 2). They are places where what the agent *asserts*, in its prompt or its plan, diverges from what the environment *permits* and not cases of the search simply failing. The rest are bookkeeping across boundaries (Table 2) and one edge left unlearned. We offer the taxonomy as a checklist for harness work: legality at revisits, death as falsification, batch boundaries, merged-state provenance, and advisory honesty.

Table 2: Defects found by the red teams in the Duck line, listed under the version that fixed them (each was found in the preceding version and fixed with a regression test and a mutant).

version	**main defects found**
v2	life counter inherited across a level boundary (spurious reset)
v3	click written into a shared enum; certificate surviving the agent's death; plan crossing a level boundary in one batch
v4	action outside the legal list on a revisited state; certificate re-issued on the game-over frame after a reopen or reindex
v5	invented action list when only RESET was legal; mask advisory claiming removals that had not happened; "proven no-op" hint in a falsified level; hidden death on a change or progress edge, or at the life budget, not marking the level
v6	unlearned edge into the RESET-only state (loop); BFS over an illegal edge inherited from a merge; "proven" hint for a no-op inherited from another frame
v7	"not yet tried" hint outside the legal list; orphaned start state after reindex; BFS through a state merged from two screens under an action legal on only one

5 Experiments

Leaderboard (official). Scores are in percentage points (0–100), as on the leaderboard. Our first scored submission (Duck fork + K, version v5 of the line) completed at **0.75** (leaderboard ref. 57852519). Our measure of the other base is the intact dfranzen notebook, which completed at

26.74 (ref. 57881859), inside the 25.8 ± 3.9 spread of unchanged copies submitted by other teams, as reported by Lam Dang (2026). Version v14.1 (last-resort mode plus the cross-game memory) was submitted on 2 November at 22:30 UTC. Its hidden-set rerun, capped at 9 hours, was completed after the competition deadline, so it is not an eligible entry and does not appear on the leaderboard. Its score, **28.41** (ref. 57923098), is reported here as a measurement from one draw, 1.67 points above our draw of the intact base and inside the base's reported leaderboard spread. No effect is inferred, and since the memory was not exercised in phase A, the score cannot be attributed to it. Two of our entries were therefore eligible, of which the intact base (26.74) is the team's leaderboard score, and the first, 0.75, is the only eligible entry that carries our agent.

Phase A (public games). Each candidate version first ran the three public games inside the Kaggle kernel. Table 1 gives the certificate counters and hidden-state levels of v4 and v5, including one level marked hidden by trigger (ii) with zero certificates (inferred). There was one draw per version, and the line's phase-A spread was not measured. In the ablation below, arms with three valid rounds vary by 2.6 to 11.7 points (sample SD) from round to round. A single phase-A difference between two versions is therefore not evidence about the explorer or its certificates.

Three modes of presence (the central comparison). Under RHAE's per-level `(h/a)^2`, every action counts, the explorer's included. (Mode 1, *playing.*) In the hybrid configuration K took most turns, and in phase A of v5 one game had 210 model turns against 370 K turns and spent 857 actions, 770 of them on a level it did not close. The scored submission landed at 0.75, between pure K on the public games (≈0.4 on the local bench described below, a different game set) and the intact dfranzen base on the leaderboard (26.74). The two bases differ, and no paired control of the Duck base was scored, so this is evidence compatible with the explorer's action cost, not an estimate of its effect. Separately, on the same dfranzen base and public games, two phase-A rounds with K playing (v9.1 code) scored **17.08** and **38.16** (mean 27.62), against 72.32–77.43 for the base. These two rounds are descriptive (n = 2, under the three-round rule) and are not a pairing of the leaderboard scores.

(Mode 2, *advising.*) K then stopped playing and instead observed every model transition, issued certificates, pruned certified keys from the model's list, and annotated the prompt. In its first phase-A round (three public games, 2,700 s each) the advisor issued 28 certificates and made 57 key removals (summed over turns). It recorded zero falsifications, which here is not evidence, since the model never executed a certified (state, action) pair (model_act_certified = 0), and a certificate is falsified only by executing it. The ablation compares the advisor with the unmodified base under matched settings (same games, caps, and concurrency, with server start-up not controlled). Only rounds that passed the validity checks are read, and one base round, lost because the model server never became ready, was rerun. The base scored **77.43, 74.98, 72.32** (mean 74.91, sample SD 2.56) and the advisor **68.85, 59.42, 76.86** (68.37, 8.73). We fixed the reading rule after a first pilot round of the base and the advisor (77.43 and 68.85) and before collecting the further rounds, and the reported means retain that pilot. The rule asks for three rounds per arm and reads an effect only if the difference of means exceeds twice the within-arm SD. After the rounds, we read that SD conservatively as the larger sample SD of the pair, and by that rule the advisor shows no measurable effect (6.5 points against a bar of 17.5). The advice-text arm without pruning has three valid rounds (**42.09, 53.55, 65.57**, with mean 53.74 and SD 11.74), after one further round, the second, was excluded for an exception in our layer. The arm lies below every base round but also under the bar (21.2 against 23.5), so it shows no measurable effect, which is not the same as no cost. Across the nine valid rounds the spread is large in every game (ls20 8.46–53.57, tu93 46.67–100, vc33 28.35–100), and this is the matched-condition evidence of the paper (Figure 3).

We diagnosed the largest gap between the two pilot rounds, in one game (ls20). The model's visible reasoning never mentioned the advisory notes, and its two expensive oscillations, strategies the model itself wrote, called actions directly in a code loop that bypassed the pruned list. This does not exclude an effect of the notes, or of pruning that the prompt did not name, elsewhere in the run, but we could not attribute the gap to the advisory layer from this trace.

(Mode 3, *last resort.*) The final configuration (Section 3.1) keeps prompt and action list identical to the base and lets K play only in the final 600 seconds of each game's cap. Nothing in this construction makes the mode additive, since the window replaces the model's turns, and the model sees the explorer's actions in its history. The construction caps the intervention in time and in actions. The loss of score is not capped and is unmeasured. The 600-second window was 22% of a 2,700-second phase-A game and is about 2% of the hidden-set cap, so phase-A runs of this mode (72.62 over three games in v14.1) are not comparable with the base. The comparable window (the same 1.88% fraction, 50.75 s) was fixed after those runs, and its hidden-set draw is the 28.41 reported above, against 26.74 for the intact base.

Second batch (7 November). After the competition, we ran the four public notebooks of Section 10 as a second batch on the same base, games, and caps (2,700 s per game, three games concurrently), with the control arm run on the same day as the other arms. The control scored **41.28, 41.35, 52.38, 74.18** (mean 52.30, sample SD 15.49). The fourth control round was a re-push that carried the notebooks' revised text, and we declared before its result that every valid control round would be read, so the base has four rounds and each mode three. The spread within one day is as large per game as in the first batch: across the four control rounds vc33 ranged from 21.43 to 100.00, tu93 from 32.08 to 100.00, and ls20 from 16.97 to 35.71, and the round means from 41.28 to 74.18. Under the reading rule above, an arm registers an effect in this batch only if its mean differs from the control's by more than twice the larger sample SD of the pair, which here is 45 to 48 points, since every arm's SD exceeded 15. K playing scored **21.03, 64.06, 61.35** (mean 48.81, SD 24.10) and shows no measurable effect. In its first round K took 833 of 858 turns in ls20 through the time-pressure trigger, and that game scored 0.00. In the second it took 346 of 414 turns in ls20, and the round scored 64.06. In the third it took 297 of 381, and the round scored 61.35 (Table 3). The first two rounds are 43 points apart, which is the reason no single phase-A round is read as evidence. K advising scored **50.80, 29.06, 74.04** (mean 51.30, SD 22.49), with no turn taken by K, and shows no measurable effect. The last-resort arm ran with the comparable window of 50.75 s (1.88% of the 2,700 s cap, the fraction of the hidden-set rerun) and scored **39.51, 81.64, 39.78** (mean 53.64, SD 24.25). K executed 79, 45, and 63 actions inside the window (summed over the three games), and the arm shows no measurable effect. One further last-resort round (65.26) was invalidated by nine exceptions from our layer in vc33 and rerun, and its replacement is the third round read. This is the first phase-A measurement of that mode under a window comparable with the base. With three or four rounds per arm and SDs above 15, no effect smaller than about 45 points would have been read in this batch.

Cross-game memory (v14.1, phase A). The store received two registrations (family G2, from ls20) and changed the leader 82 times in ls20, a count that includes model turns. No K decision was changed (`k_cross_decisao` = 0 in all three games), so by the pre-fixed rule the arm was not exercised and we draw no conclusion. In vc33 the model closed all seven levels before the last-resort window opened.

Method lessons from phase A. Phase-A comparisons are fragile in two ways we did not control at first. First, with six games on one GPU, the model took 40–74 turns per game, against 49–101 with three games, in runs that also differed in window and configuration. This is therefore an observation

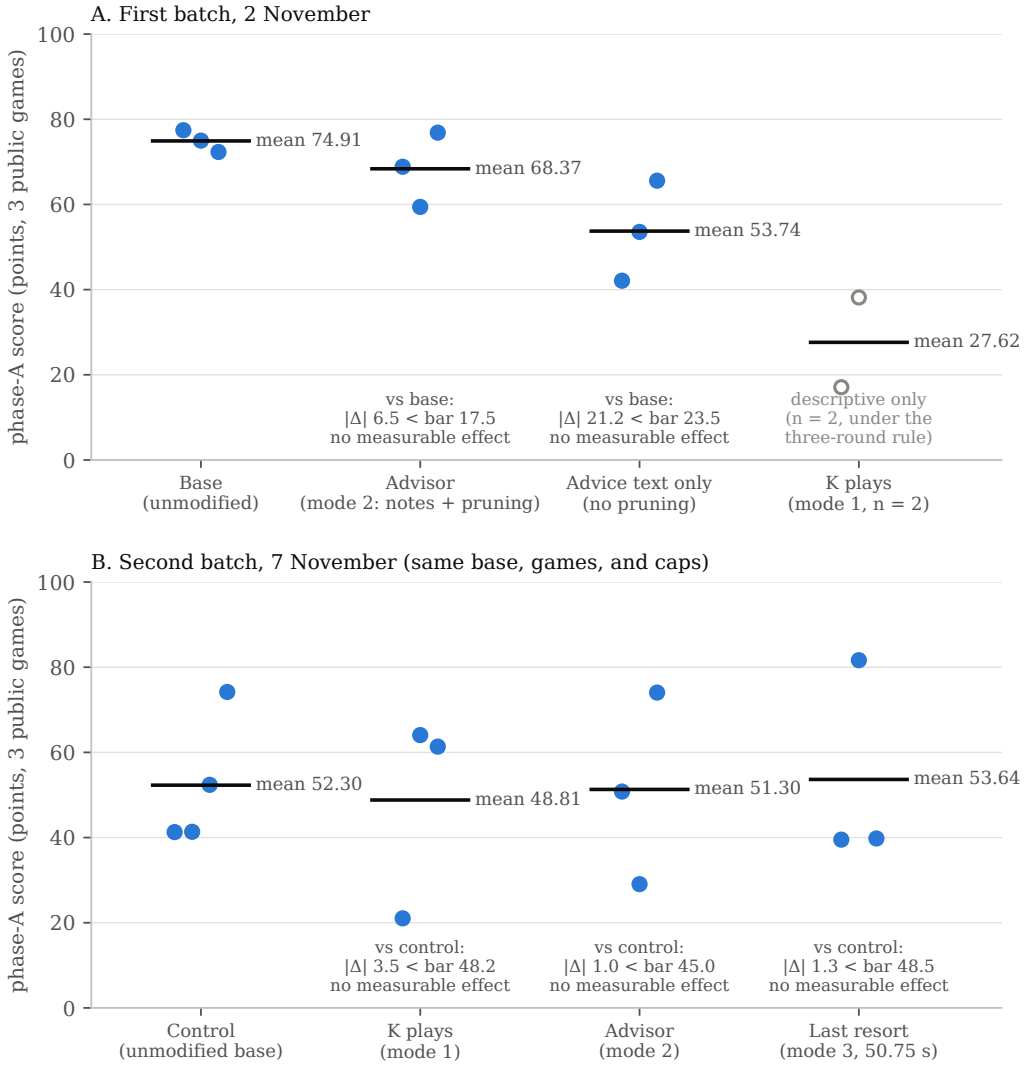


Figure 3: Phase-A rounds per arm on the same dfranzen base, games, and caps (three public games; one dot per valid round; black ticks are means; hollow dots mark an arm with fewer than three rounds, descriptive only). A, first batch, 2 November. B, second batch, 7 November, with the control run on the same day as the arms. The rule, fixed after a first pilot round of the base and the advisor and before the further rounds, reads an effect only if the difference from the base exceeds twice the within-arm SD, read conservatively as the larger sample SD of the pair, and no arm passes it in either batch. In A, the K-plays arm has two rounds and is not a pairing of the leaderboard scores. In B, every arm has three rounds and the control four, and the bar is about 45 points, since every arm's SD exceeded 15.

Table 3: Second batch, phase A on the dfranzen base (three public games, 2,700 s cap per game, run concurrently; RHAE in percentage points; actions summed over the three games). An arm is read when it has three valid rounds. Validity per round: module hashes equal to the frozen manifest, no traceback, no exception from our layer, and non-zero K counters in the arms with K. The Duck-fork run of the same day is not in this table, since the bases differ.

arm	round	mean	ls20	tu93	vc33	actions	note
control (unmodified base)	1	41.28	22.41	80.00	21.43	752	
control	2	41.35	16.97	32.08	75.00	871	
control	3	52.38	35.71	100.00	21.43	831	
control	4	74.18	32.55	89.99	100.00	1,037	text-only re-push; read (declared before the result)
K playing	1	21.03	0.00	9.52	53.57	1,734	K took 833 of 858 turns in ls20
K playing	2	64.06	12.18	80.00	100.00	1,516	K took 346 of 414 turns in ls20
K playing	3	61.35	30.18	53.89	100.00	1,531	K took 297 of 381 turns in ls20
K advising	1	50.80	28.91	78.63	44.87	913	no K turn
K advising	2	29.06	32.95	33.16	21.08	785	no K turn
K advising	3	74.04	22.12	100.00	100.00	1,068	no K turn
last resort (50.75 s window)	1	39.51	34.88	62.22	21.43	920	79 K actions in the window
last resort	2 (invalid)	65.26	30.80	64.99	100.00	1,045	nine exceptions from our layer in vc33; not read; rerun as round 4
last resort	3	81.64	44.92	100.00	100.00	976	45 K actions in the window
last resort	4 (rerun of 2)	39.78	35.71	55.64	27.99	988	63 K actions in the window

and not a measured slowdown. Second, the model server's weight-loading stage took 305 to 1,494 seconds across eight phase-A runs (in four of them the games were released while the server was still loading). This is consistent with loading a ~190 GB checkpoint from a network filesystem, which copying shards to RAM reduces to about 300 seconds in an interactive session (Lam Dang, 2026). In one measured round of the unmodified base, model inference occupied 81% of game time, and we did not profile the remaining ≈19%, which includes a fresh sandbox per call and the whole history serialized per action. We report phase-A deltas only under matched concurrency and matched caps. Because only a scored submission can be selected, a last submission needs the full rerun budget (here 9 hours), plus margin for queue and setup, before the selection deadline. Ours did not have it, and another team on the same base reports missing selection the same way (Lam Dang, 2026). A third fragility appeared between batches. The control arm of the second batch, run on 7 November with the same phase-A code as the first (equal abstract syntax tree to the intact-sample notebook), scored 52.30 on average over four rounds against 74.91 on 2 November, about 23 points lower, with a within-day SD (15.49) six times the first batch's (2.56). The two batches differ in at least one measured condition, since on 7 November the model server was ready one to two minutes after the games started, whereas on 2 November loading took 305 to 1,494 seconds. We have not measured the cause, we do not pool the batches, and every comparison in this paper is read within the batch in which it was run.

Exploration-first, measured alone. Without any language model, a local bench (25 public games, 1,500 steps, 4 seeds, scored by the toolkit's RHAE in the leaderboard's percentage points) measured the lineage on which K is built. That lineage starts with graph exploration and then adds the two fixed goal heuristics of Section 3.1. The runs are logged from 29 September to 3 November. The code of runs before 26 October is not versioned, and only the current K without H5 (K0, Table 4) was rerun. Replacing an operational-memory agent with a state graph and BFS to the untested frontier raised completed levels from 9.5 to 14.25 of 183 per seed (paired, same seeds). In a same-day ablation on the same seeds, removing BFS from its successor more than halved the completed levels ($15.5 \rightarrow 6.75$), but the score barely moved (0.273 vs. 0.260). On the levels it completes, the lineage takes a median of about 13 times the human baseline actions, both before the goal heuristics and after them. Only 3–8% of its completions are at or under the human count, and because the per-level score squares the action ratio, the others are worth almost nothing. It plateaus at about 16 of 183 levels and a score near 0.39 on two seed sets. K has none of the machinery of novelty search, the

method of Lehman and Stanley that rewards diverging from prior behaviors instead of progress toward the objective, and it has no archive of past behaviors and no sparseness measure. The cost measured here therefore belongs to our explorer rather than to the method of Lehman and Stanley. They answer the objection that novelty search is exhaustive search by noting that the behavior space may often collapse into a manageable number of points (2011, pp. 198–199). They list among its limits that it has no bias toward optimization once a solution is found (p. 213), and a per-action score charges that limit. They suggest that it may be more efficient to search for novelty first and tune by the objective after (p. 213), the reverse of the goal-first premise of Section 3.3. The bench measures the explorer alone on the public games and not the layer inside a language model base, so it is consistent with mode 1 and does not identify its effect.

Local bench (H5 attribution). Eight arms ran on the same bench, K alone, on the submitted K file, and each arm switches families or ordering through the bench's flags. The arms are read by a rule recorded while they were running, after the K0 arm had finished. Under it, an effect needs a paired mean difference beyond twice the default arm's between-seed SD. We later took the population SD over four seeds (0.0817, so the bar is 0.1634), and with the sample SD, the bar would be 0.1886. No arm passes either (Table 4), and the largest difference is 0.0515. The identification counts are the result we emphasize (Figure 4). With the default families, 50 of 66 wins have no true hypothesis instance in the winning frame (49 of 66 in the version first measured), and the leader before the win was true in 2. We call these wins anomalies. The winning frame is the last frame observed before the level change, and the instances checked are those not yet eliminated (expired ones included). "True" here means that a predicate we implemented holds in the frame, not that it is the game's goal, and a win is ambiguous when more than one instance is true. Removing G2 coincided with a drop in wins with a true instance from 16 of 66 to 5 of 64 in this bench, where trajectories also changed, so this is a coverage difference and not a paired decomposition. Re-enabling O1 lowers the anomalies to 19 of 63 but marks 327 true instances in 63 wins, 43 of them ambiguous, with no score gain, so it measures the breadth of a proximity predicate and not identification. A fixed random priority per instance leaves 48 of 63 anomalies, so ordering does not explain most of the missing matches, though it may matter in particular games. On an 8-seed paired bench, earlier H5 generations measured -0.062 ± 0.025 (v9) and -0.006 ± 0.028 (v9.1, O1 off) against pure K (mean paired difference $\pm$ standard error over the eight seeds).

Table 4: H5 attribution bench (25 public games $\times$ 4 seeds $\times$ 1,500 steps; K alone). RHAE is in percentage points, as on the leaderboard. Wins are levels completed; wins and counts are summed over seeds; SD is the population SD over the four seeds; ambiguous wins have more than one true instance. K0 keeps the H1/H2 guides, which H5 replaces with its leading instance. "Without G4" matches the default score with different trajectories, absorbed by the 1.15 cap.

arm	**RHAE $\pm$ SD**	**wins**	**no true instance**	**ambiguous**
K0 (no H5; H1/H2 guides)	0.3808 ± 0.1042	65	—	—
default (O3, G1, G2, G4)	0.3641 ± 0.0817	66	50	10
without O3	0.3674 ± 0.1004	66	54	10
without G1	0.4156 ± 0.0811	67	52	9
without G2	0.3287 ± 0.1122	64	59	0
without G4	0.3641 ± 0.0817	66	49	10
default + O1	0.3198 ± 0.0903	63	19	43
randomized order	0.3557 ± 0.0952	63	48	9

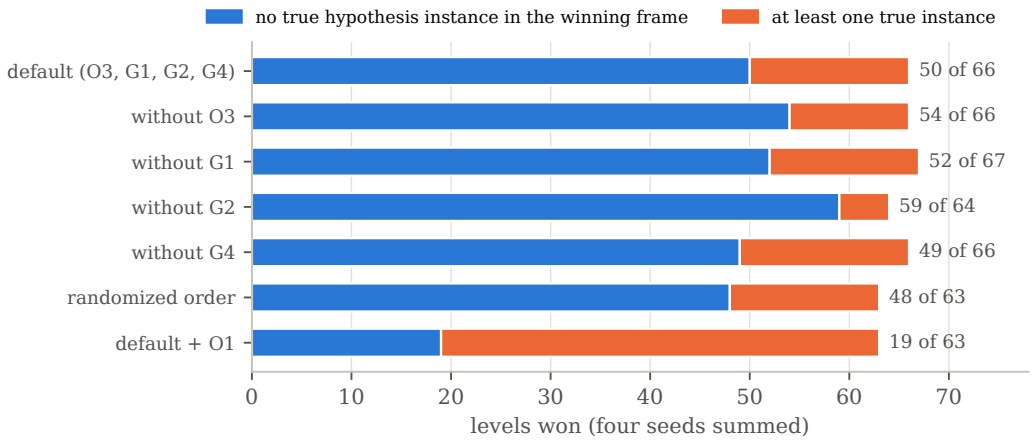


Figure 4: H5 attribution bench (Table 4): levels won per arm, split by whether any implemented predicate instance is true in the winning frame. "True" means that a predicate we implemented holds, not that it is the game's goal; with O1, a broad proximity predicate, most wins contain true instances (43 of them ambiguous) without any score gain.

6 Transfer beyond ARC-AGI-3

Three parts are not specific to this benchmark's games. A no-effect certificate needs only a discrete action set and an observable successor, together with a stable way to say "the same state" (here, a masked hash), and falsification and revocation are bookkeeping. Hidden-state detection by certificate failure is, in general form, a test for perceptual aliasing (two situations sharing one observed state) that any interactive agent can run on its action log. The adversarial protocol of Section 4 applies verbatim to any versioned agent artifact. The three modes transfer only as a design hypothesis. Where an auxiliary explorer's actions enter the main agent's score, letting it play is expected to be costly under per-action scoring, since alone our explorer spends about 13 times the human actions per completed level (Section 5). In our ablation, advising had no measurable effect.

What does not transfer is the move. Each game fixes one goal and varies its parameters from level to level, and it never states the rules that make some moves valid and others impossible. In Ackoff and Emery's terms, as we read them, a game's levels are structurally different environments for one goal, and different games deal with different goals (1972/2017, pp. 30–31). An edge is a fact about one game under its rules, and so are a certificate and a route, which is why the memory of Section 3.4 carries only a family or a schema of roles. Churchman (1971, pp. 125–126) imagines observers who examine completed games of ticktacktoe "to detect the common terminating characteristic" and a mathematician who represents the same game as filling a matrix "so that it satisfies some property." When the others complain that he has "introduced too much of his own background," he replies that they introduced theirs as well, and that his representation "can be used to induce rules for many other games." Our goal families play a similar role, a representation brought from the designer's background. Where the mathematician claims that reach, we have not tested ours beyond this environment family. The 50-of-66 count is what that background misses here. K's life budget is learned within each game, and the rule that infers it was chosen and tested only on the 25 public games, like K's other constants and the last-resort caps.

One lever, the per-level action count, runs through all three modes. After the deadline, in the

sixth-placed participant's thread, the fourth-placed participant asked whether action efficiency, "not tokens to level but actions to level," was "one lever we missed." The thread's author agreed, and the fourth-placed participant reported that including the score formula in the prompt sometimes made the model "just give up" (Zatravkin, 2026b). In the sixth-placed team's rehearsals on the 25 public games, extra moves consumed 15.2–20.1% of won-level points at the end of three 9-hour runs (11.8–16.1 points per copy against crediting every win at no worse than the human action count). In the same runs, 30–34% of wins used more actions than the human baseline (Zatravkin, 2026a). Our certificates act on that lever at the action list, but in the game we diagnosed, the costly moves came from the model's code, which bypassed the list (Section 5). We have not shown that our notes or our list changed the model's action counts.

7 Limits and negative results

- (i) The explorer, which learns no world model, plateaus at about 16 of 183 public levels per seed and a median of about 13 times the human actions per completed level. That cost was measured with the explorer alone, outside any language model base.
- (ii) Win-condition identification, as implemented, is weak where it matters, since in 50 of 66 wins no true hypothesis instance is present in the winning frame. This resembles a second-order failure in Bateson's sense, choosing from the wrong set of alternatives (1972, p. 286). The implemented witness set did not match the winning frame, and changing rank alone cannot create a missing witness.
- (iii) No H5 arm shows a measurable score effect on the attribution bench, and the earlier generations measured slightly negative to neutral. Lehman and Stanley report a similar outcome in a maze without outer walls, built to test the limits of their method. There, novelty search found the goal only five times in a hundred runs, and objective-based search fared no better, with two (2011, pp. 204–205). Likewise, replacing K's fixed goal heuristics with H5's families did not make the search succeed. We submit the component for its measurement value and make no score claim for it.
- (iv) Hidden-state marking by trigger (ii) fired once with zero certificates. Whether marking a level hidden-state on that evidence alone is the right response is an open design question, which we report rather than settle.
- (v) Mode 1 compares different bases and is evidence compatible with the explorer's action cost, not an estimate of it. On the same base, mode 1 has two descriptive rounds in the first batch and three in the second, where it shows no measurable effect against a same-day control (Table 3). Mode 2 shows no measurable effect for the advisor in either batch, or for advice text alone in the first (three valid rounds per arm), though every advice-text round of the first batch lies below the base. Mode 3 caps the intervention by construction, but not the score loss. Its comparable window was measured in phase A in the second batch, with no measurable effect, and its one hidden-set draw (28.41 against 26.74, scored after the deadline) is inside the base's spread.
- (vi) The cross-game memory was submitted and not exercised in phase A, and its trajectory depends on thread timing.
- (vii) Leaderboard numbers carry draw noise (SD 3.9 across unchanged copies of the base submitted by other teams, mean near 26), and each submission yields a single leaderboard

draw. Version-to-version leaderboard deltas are therefore weak evidence, and we do not argue from them. Our phase-A and bench spreads are reported where measured. Unchanged bundles resubmitted by the sixth-placed team drew 22.75/23.46 and 21.48/26.06 on the public leaderboard (Zatravkin et al., 2026). Two batches of the unmodified base on the same public games, five days apart, differed by about 23 points in mean (74.91 against 52.30), so phase-A levels are not comparable across batches either.

8 Future work: not playing, not speaking, handing over the map

The advisor still speaks since it prunes a list and writes notes into a prompt. The next configuration removes both channels and hands K's graph to the model's tool as the global `k_map`. The map holds the states, the observed edges with their effects, `here`, `untried_here`, and `route_to_untried`, the shortest **known** path (an upper bound in the observed subgraph, with no guarantee of global optimality). Only observed edges enter, and edges inherited from state merges stay out. On hidden-state levels, the map reports itself unusable. It is a revocable witness (our term) at a declared level of abstraction in Floridi's sense, a finite set of observables (2019, pp. 42–45), which here are the masked frame and the tested actions. Since a level of abstraction may omit the observables that would make transitions deterministic (pp. 44–45), frame $\neq$ state is stated instead of being assumed away. The map was implemented and run in phase A on 2 November, in a version that still let K play the last-resort window. It was queried once in ls20 (after the model saw K's actions in its history) and not in the other two games, which is too little use to assess its value. In this harness, this model made little visible use of either channel, since the notes went unmentioned in the one game we diagnosed (Section 5), and the map drew that single query.

A later read-only variant, without the last-resort window, made the channel discoverable through a fixed prompt line and a short notice in the tool's return, at most once per level, for a `k_read()` call that never acts. It had no pruning and no explorer notes, counted executed calls, and ran under a validity rule fixed in advance and an independent red-team release. In one phase-A round, without a control arm, it was queried once in three games. That query came right after a reset and returned an unusable snapshot since no observation matched the current frame. Two other snippets in ls20 had incomplete telemetry and were not counted as queries. The round scored 42.70, below the three base rounds of Section 5, and with one round and no control, neither the channel nor its notices can be credited with the difference. A query shows that the channel was read and not that a decision changed. Dzotsenidze (2026) draws the same line for a recovery route that executed eight times: execution, the next action, progress, and score are separate observations, and none substitutes for another. Our low-use result is specific to these channels and this harness. In another dfranzen-based agent, one 118,173-token request put 25.0% of its sparse-attention picks on tool outputs (Zatravkin et al., 2026). That is a profile of reading rather than a profile of use.

A close antecedent of handing the record to the model's code is PRO-LONG (Fox et al., 2026, pp. 2, 4, 7–8). Its harness appends every action and board to a log that frontier coding agents search with code. In its GPT-5.5 runs, 20.3% of the tool calls other than action output went to that log, and adding regex search and then code to file reading raised the score (23.1 $\rightarrow$ 27.2 $\rightarrow$ 38.3). These are results in their setup and not a matched test of our channel.

For the graph channel itself, the arc-3 lab reports three lower-scoring state-graph runs despite heavy tool use and zero plan rejections. A lean variant exposing `untried_here` also scored lower in a seven-game A/B (1.533 with, 2.061 without; sonpham-org, 2026). The harnesses differ, so we report that result beside ours without setting the two against each other.

The foundation reports, from the other side, the split that Section 5 measures for exploration alone (about 13 times the human actions per completed level). Before launch, it had hypothesized that action efficiency would remain a dividing line between humans and AI. That, it now writes, "remains true of brute-force approaches," and once frontier AI "understands" the mechanics, it "generally executes within the range of human efficiency" (Kamradt, 2026b). The distinction is older than the benchmark. Harlow described the learning sets of Section 3.4 as transforming an animal "from a creature that adapts to a changing environment by trial and error to one that adapts by seeming hypothesis and insight" (1949, p. 51). Tolman, studying how rats learn mazes, separated a narrow "strip" map, which links position to goal "by only a relatively simple and single path," from a comprehensive map of the wider environment. The difference shows only when the environment changes, and the narrower the map, "the less will it carry over successfully to the new problem" (1948, p. 193). Among its causes, he listed "an overdose of repetitions on the original trained-on path" (pp. 205, 207). For them, these are states of an organism. Here they describe what our explorer's graph is, a record of one level's routes, and they motivate the first of the two arms below, which asks what carries over to level k+1.

Two further arms, not yet run, would be read by the ablation's rule (three rounds per arm; twice the larger within-arm SD). The first is a cross-level mechanics ledger, motivated by the expectation that a game's first level demonstrates the mechanics its later levels elaborate. It is measured as actions to first progress in level k+1 with and without the ledger, a rate counted in trials with the goal held fixed. That is how Ackoff and Emery measure intelligence, as the rate at which a subject learns (1972/2017, pp. 52–53), here extended to a system below the purposeful subjects for whom they define it (pp. 34, 52). The measure includes the negative side, the delay when the mechanic changes. When Harlow's problems switched without warning from rewarding an object to rewarding a position, the monkeys' established learning set first held back the new learning (1949, pp. 60–61). Bateson predicts this case, since when earlier Learning II misidentifies the contingency pattern of a new context, "we shall expect a delay of Learning I in the new context" (1972, p. 294). The arm splits into per-object effect ordering (M2a) and reused macro-actions (M2b), in the sense of Sutton, Precup, and Singh's options, closed-loop policies that, once started, run over several steps until their stopping condition (1999, pp. 181, 186). The second arm is family suspension in the goal component, a revision of the set of families (Learning II in Bateson's sense, 1972, p. 293; Section 3.4). It is decided by a second-order signal (uniform credit across families, or revisions growing without level gains) rather than by each family's progress scalar. Suspensions and returns are both counted, and suspension never deletes.

Three directions follow from the limits above. The first concerns connectors, since our layer reached the model only through the prompt, the action list, and, in the later variants, a queryable map. In the game we diagnosed, the model's code routed around the list (Section 5). A harness built to consume such claims, or a model fine-tuned to read them, would tell whether the claims fail or only their channel. We refined neither the model nor the harness for this. The second concerns the hypothesis space, where the goal families were authored by us and 50 of 66 wins fell outside them. Families induced from level transitions, in the line of EMPA and of the Misfit agent (Section 2), would move part of that authorship from the developer to the agent. The third concerns probes, as the conjecture of Section 1 can be tested on the transcripts we keep, by counting the turns in which the model states competing hypotheses and the actions it takes to tell them apart.

We close with a suggestion about benchmark design, offered within the limits of a single study. The largest gains reported in this competition came from the model and its serving, with the agent left almost unchanged. Expert pruning, concurrency, faster loading, and scheduler fixes raised one team's

leaderboard score from about 26 to 33.68 and then to 37.75 (Lam Dang, 2026), and the sixth-placed participant called serving optimization "the BIGGEST thing to improve," and harness changes "died in the noise" (Zatravkin, 2026b). Outside the competition, a frontier model's score rose from 62.7% to 99.9% with a harness built on its provider's context-management features, at a reported cost of about $19,000 for that run, against $26,000 for the standard-harness run (Kamradt, 2026b). These outcomes are legitimate, and as we read them they nonetheless reward compute and the engineering of inference. Two reports qualify the leaderboard itself. One team's identical code scored between 43.18 and 53.27 across its runs, a spread of 10.1 points, "so single draws cannot separate most changes" (Tanaka et al., 2026). The same team banks its wins: because the scorecard scores a game by its best play, after a WIN its harness sends RESET and replays the winning sequence without its exploratory moves, which raised its local 110-game mean from 48.3 to 59.3. In the comments, the second-placed team reports that on 825 recorded wins the replay raised the mean per-game score from about 67 to 92, an aggregate gain "on the order of +12 points" "with no improvement to the agent's actual solving ability," and left it unconfirmed on the competition gateway (Tanaka et al., 2026). A replayed win is scored as a play, and the score cannot tell it from a first one. The quantized checkpoints that both bases serve were prepared by third parties (Section 3.1), and one team's attempt to adapt the model by reinforcement learning ran for about two days on a GPU larger than the competition's, without a run long enough to measure an effect on play (Lam Dang, 2026). The public first place reports the same direction carried through with the resources it takes: expert pruning and INT3 quantization recovered by distillation, a private serving stack on an RTX PRO 6000 for 110 concurrent sessions, and reinforcement learning on eight B200 GPUs aimed at the stages the base model could not solve (Chen, 2026). The training raised hard-stage solving but lowered action efficiency, with runs above the human action count rising from 16% to 26%, so the final submission kept the base model for the first six hours and switched to the trained one for the last three. That is the shape of our last-resort mode, reached from the other end. We kept the base and added an explorer, which trains nothing; the winning entry kept the base and added a trained model and its serving, which cost the hardware that the dedicated track below would fix across entrants. None of this tests whether an agent identifies a goal, let alone chooses one. Much of it is also fitted to the competition's execution environment (its model-loading latency and its wall-clock cap), so that more tokens per second buy more inference per game (Section 2; Section 5, method lessons). That is a valid way to compete, but what it measures is throughput in one environment, whereas the efficiency that the foundation's definition of AGI names is the efficiency of acquiring a skill (ARC Prize Foundation, 2026, p. 4). Chollet set the information efficiency that his definition measures apart from computation and time efficiency, which he listed as alternatives that a definition could add (2019, p. 41). Two systems could differ widely in such optimization and both meet that definition, as language models of different lineages and speeds are all called AI.

The score therefore steers entrants toward the resources that make play cheaper. The foundation asks for ideas. Hooker (2021) calls it a hardware lottery when a research idea wins because it is compatible with the available software and hardware, "not because the idea is superior to alternative research directions" (p. 58), and asks which failures "we still don't have the hardware to see as a success" (p. 63). ARC-AGI-3 is not meant to measure "the amount of human intelligence that went into designing" a system built for it (ARC Prize Foundation, 2026, p. 15), and open-ended innovation is among the questions for its next benchmarks (Kamradt, 2026b). We read open-ended innovation, in this setting, as an agent producing a goal or a strategy that its designers did not write into it, and a benchmark can make room for it in two ways. First, it can score each component it names apart from action efficiency. For goal-setting, the agent would state the win condition it pursues before reaching it, and the statement would be scored against the game's actual condition.

An identifier like ours, which did not name the goal in 50 of 66 local wins, would then be measured directly instead of only through actions. Second, it can fix the model and its serving across entrants in a dedicated track, so that score differences come from the agent and not from compute. Such a track would not deny Sutton's bitter lesson, that "general methods that leverage computation are ultimately the most effective, and by a large margin" (2019, para. 1). It would measure the agent apart from that lesson instead of forbidding it. The agents Sutton asks for are ones "that can discover like we can, not which contain what we have discovered" (para. 8), and a score for goal-setting would measure that discovery. Dehghani et al. (2021) make the same case for a cap on tuning budgets, which would give "comparisons that are tied less to the computational budget of the proposing entity, but more to the merits of the methods themselves" (p. 16). Every benchmark, as they put it, "makes a statement about what it perceives to be important" (p. 1). A benchmark whose score is aligned with all the dimensions it names would give ideas that do not yet raise today's score a measure of their own.

9 Conclusion

ARC-AGI-3 evaluates goal-setting, yet each game fixes its win condition, and the score measures how economically it is reached. In Ackoff and Emery's taxonomy, the step that competing agents take is the identification of a goal that the environment determines, which places them, ours included, at most among multi-goal-seeking systems. By our working definition, a general intelligence would also select its goals (Section 1). In 50 of 66 local wins, none of the goal predicates we implemented held in the winning frame, yet the explorer still won by search (Section 5). The result is consistent with what other designs report from different directions, as in Twin, where dynamics are learned far more reliably than the goal (Skoutnev et al., 2026, p. 28). In another entry, an explorer with a learned forward model scored no better than a random policy on the public games, and its author concludes that a feature predictor that learns movement well "can still lack the representation needed to identify a goal" (Shamim, 2026, pp. 11, 14). In DiG-bench, a brief description of the rules and the win condition raised one model's wins from 18 to 69 of 70 games (Battleday et al., 2026, p. 6). Without a world model and with a different measure, our result points to the same component.

The second finding concerns reach, since none of the three modes of presence (Section 5) produced a measurable score gain. In the game we diagnosed, the language model's costly moves came from its code rather than from the layer's claims (Section 5). Identifying the win condition and making that identification reach the actions is where we place the open part of goal-setting, one of the four components across which the benchmark defines intelligence as efficiency. Choosing goals, the condition that our working definition adds, lies beyond what this benchmark asks and beyond what we tested.

10 Reproducibility

The public notebook ("What Exploration Can Ignore: Duck fork, K playing," `https://www.kaggle.com/code/hralmjc/what-exploration-can-ignore-duck-fork-k-playing`) is a clean copy of the code of our eligible entry that carries the agent (ref. 57852519, the Duck fork with K playing, mode 1). Its code cells and embedded files are identical to the submitted kernel after renaming, checked by abstract-syntax-tree comparison, with the internal logs removed, and it was not re-scored on the hidden set. The three modes on the dfranzen base are published as notebooks with the same frozen code, namely K playing (`https://www.kaggle.com/code/hralmjc/what-exploration-can-ignore-dfranzen-k-playing`), the unmodified base as control (`https:`

//www.kaggle.com/code/hralmjc/what-exploration-can-ignore-dfranzen-control), K advising (https://www.kaggle.com/code/hralmjc/what-exploration-can-ignore-dfranzen-k-advising), and last resort (https://www.kaggle.com/code/hralmjc/what-exploration-can-ignore-dfranzen-last-resort), which differ from the frozen artifacts only in how they locate their inputs on Kaggle, as declared in the manifests. Kaggle mounted the same inputs under two different layouts for different kernels of this set, and the notebooks resolve both and record which one they found. The second batch of Section 5 (Table 3) ran these four notebooks. The later dfranzen + K kernels, including v14.1 (last-resort mode plus the cross-game memory, whose trajectory depends on thread timing and is reproducible in code, not in run), are identified by frozen artifacts and SHA-256 manifests, like the scored Duck-line submission. Local-bench numbers come from a separate bench harness (25 public games, offline, scored by the toolkit's RHAE) that is not part of the submitted notebook. A reproducibility package, released in a public GitHub repository (https://github.com/hralm/arc-agi-3-paper-track-2026) together with this paper, contains the agent code (K, H8, and H5, including the cross-game memory), the bench harness with the flags of the eight attribution arms (Table 4, Figure 4), the test suite with the protocol's mutants (Section 4), the SHA-256 manifests of the frozen artifacts, and the scripts that generate the figures. It also holds the five notebooks above, with their kernel metadata and embedded modules, and, for every phase-A round reported here (Tables 1 and 3, Figure 3), the files from which the numbers were read, the kernel log, the harness summary, and the benchmark record, with a script that recomputes the validity checks and the tables from them or from the output of a new Kaggle run. A reader who clones the repository has every component cited in this paper without visiting each Kaggle page. The development sessions' working notes, and the model transcripts and request logs of the runs, are not released. The protocol itself is described in full in Section 4.

References

Ackoff, R. L., & Emery, F. E. (2017). *On purposeful systems: An interdisciplinary analysis of individual and social behavior as a system of purposeful events*. Routledge. (Original work published 1972)

ARC Prize Foundation. (n.d.). *ARC Prize 2026* [Documentation]. Retrieved September 29, 2026, from https://docs.arcprize.org/arc-prize-2026

ARC Prize Foundation. (2026). *ARC-AGI-3: A new challenge for frontier agentic intelligence* (Version 2, arXiv:2603.24621). arXiv. https://doi.org/10.48550/arXiv.2603.24621

Bateson, G. (1972). The logical categories of learning and communication. In *Steps to an ecology of mind* (pp. 279–308). Ballantine Books. ISBN 0-345-24684-5.

Battleday, R. M., Sandbrink, K., Cullen-Drohan, J., Yan, Z., Muller, T., Maguire, C., Kubicek, A., Greenlee-Scott, F., Sumant, S., Dao, T., Schmidhuber, J., Valko, M., Tenenbaum, J., Griffiths, T. L., Kurth-Nelson, Z., & Whittington, J. C. R. (2026). *DiG-bench: Discovery in games* (Version 1, arXiv:2608.12593). arXiv. https://doi.org/10.48550/arXiv.2608.12593

Bessis, H., Cottaar, J., Pressman, I., Smit, D., Tešnar, M., & Viel, S. (2026a). *Tufa Labs duck harness [June 30 milestone winner]* [Kaggle notebook]. Kaggle. Retrieved September 30, 2026, from https://www.kaggle.com/code/jeroencottaar/tufa-labs-duck-harness-june-30-milestone-winner

Bessis, H., Cottaar, J., Pressman, I., Smit, D., Tešnar, M., & Viel, S. (2026b, July 1). *Tufa Labs' winning solution for ARC-AGI-3 Milestone 1* [Online forum post]. ARC Prize 2026 – ARC-AGI-3, Kaggle. Retrieved November 9, 2026, from https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/discussion/717133

Chen, Y.-C. (2026, November 4). *[ARC-AGI-3] Public 1st place solution: Quick overview* [Online forum post]. ARC Prize 2026 – ARC-AGI-3, Kaggle. Retrieved November 9, 2026, from https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/discussion/755795

Chollet, F. (2019). *On the measure of intelligence* (Version 2, arXiv:1911.01547). arXiv. https://doi.org/10.48550/arXiv.1911.01547

Churchman, C. W. (1971). *The design of inquiring systems: Basic concepts of systems and organization*. Basic Books. ISBN 0-465-01608-1.

Courtis, D., Li, W., & Sanner, S. (2026). *OPINE-World: Programmatic world modeling with ontology-error-prioritized interactive exploration for ARC-AGI-3* (Version 3, arXiv:2607.01531). arXiv. https://doi.org/10.48550/arXiv.2607.01531

Dehghani, M., Tay, Y., Gritsenko, A. A., Zhao, Z., Houlsby, N., Diaz, F., Metzler, D., & Vinyals, O. (2021). *The benchmark lottery* (Version 1, arXiv:2107.07002). arXiv. https://doi.org/10.48550/arXiv.2107.07002

Dzotsenidze, T. (2026, November 2). *Recovery, task progress, and execution evidence in ARC-AGI-3* [Kaggle solution writeup]. ARC Prize 2026 – ARC-AGI-3, Kaggle. Retrieved November 4, 2026, from https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/writeups/recovery-task-progress-and-execution-evidence-in

Ecoffet, A., Huizinga, J., Lehman, J., Stanley, K. O., & Clune, J. (2021). *Go-Explore: A new approach for hard-exploration problems* (Version 4, arXiv:1901.10995). arXiv. https://doi.org/10.48550/arXiv.1901.10995

Floridi, L. (2019). *The logic of information: A theory of philosophy as conceptual design*. Oxford University Press. ISBN 978-0-19-883363-5.

Fox, A., Wang, J., Rosu, P., & Dhingra, B. (2026). *PRO-LONG: Programmatic memory enables long-horizon reasoning* (Version 2, arXiv:2607.20064). arXiv. https://doi.org/10.48550/arXiv.2607.20064

Franzen, D. [dfranzen]. (2026a). *ARC-AGI-3 Milestone 2 solution* [Kaggle notebook]. Kaggle. Retrieved October 26, 2026, from https://www.kaggle.com/code/dfranzen/arc-agi-3-milestone-2-solution

Franzen, D. [dfranzen]. (2026b, September 30). *ARC-AGI-3 Milestone 2: Building on the Duck harness* [Kaggle project writeup]. Kaggle. Retrieved November 9, 2026, from https://www.kaggle.com/writeups/dfranzen/arc-agi-3-milestone-2

Gharajedaghi, J. (2011). *Systems thinking: Managing chaos and complexity: A platform for designing business architecture* (3rd ed.). Morgan Kaufmann.

Harlow, H. F. (1949). The formation of learning sets. *Psychological Review*, *56*(1), 51–65. https://doi.org/10.1037/h0062474

Hooker, S. (2021). The hardware lottery. *Communications of the ACM*, *64*(12), 58–65. https://doi.org/10.1145/3467017

Kamradt, G. (2026a, March 25). Announcing ARC-AGI-3. *ARC Prize Blog*. ARC Prize Foundation. Retrieved October 23, 2026, from https://arcprize.org/blog/arc-agi-3-launch

Kamradt, G. (2026b, September 3). OpenAI's GPT-6 Astra on ARC-AGI-3. *ARC Prize Blog*. ARC Prize Foundation. Retrieved November 4, 2026, from https://arcprize.org/blog/astra

Lam Dang. (2026, November 4). *More tokens! From public NB to LB 161th (missed 42th places by a few hours)* [Kaggle solution writeup]. ARC Prize 2026 – ARC-AGI-3, Kaggle. Retrieved November 4, 2026, from https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/writeups/more-tokens-from-public-nb-to-lb-161th-missed-4

Le Grand, S. (2026). *taaf-flashnext-sheetu12b-0922* [Kaggle notebook; `agentfix` module]. Kaggle. Retrieved September 30, 2026, from https://www.kaggle.com/code/scottlegrand/taaf-flashnext-sheetu12b-0922

Lehman, J., & Stanley, K. O. (2011). Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary Computation*, *19*(2), 189–223. https://doi.org/10.1162/EVCO_a_00025

Lehmann, J., Aioanei, A., & Vahdati, S. (2026). *Tycho: Active abstraction with programmatic world models for ARC-AGI-3* (Version 1, arXiv:2607.28287). arXiv. https://doi.org/10.48550/arXiv.2607.28287

Liew, K. H. (2026). *Explore before you solve: The speed-depth trade-off in epistemic agents for ARC-AGI-3* (Version 1, arXiv:2605.25931). arXiv. https://doi.org/10.48550/arXiv.2605.25931

Maturana, H. R., & Guiloff, G. D. (1980). The quest for the intelligence of intelligence. *Journal of Social and Biological Structures*, *3*(2), 135–148. https://doi.org/10.1016/0140-1750(80)90006-8

McCree, A. (2026). *AGI in a video shop - Atom Eons Nostalgia* [Kaggle notebook; Misfit agent]. Kaggle. Retrieved October 26, 2026, from https://www.kaggle.com/code/atommccree/agi-in-a-video-shop-atom-eons-nostalgia

McGreivy, N., & Hakim, A. (2024). *Weak baselines and reporting biases lead to overoptimism in machine learning for fluid-related partial differential equations* (Version 1, arXiv:2407.07218). arXiv. https://doi.org/10.48550/arXiv.2407.07218

Rodionov, S. (2026). *Executable world models for ARC-AGI-3 in the era of coding agents* (Version 2, arXiv:2605.05138). arXiv. https://doi.org/10.48550/arXiv.2605.05138

Roli, A., Jaeger, J., & Kauffman, S. A. (2022). How organisms come to know the world: Fundamental limits on artificial general intelligence. *Frontiers in Ecology and Evolution*, *9*, Article 806283. https://doi.org/10.3389/fevo.2021.806283

Rudakov, E., Shock, J., & Cowley, B. U. (2025). *Graph-based exploration for ARC-AGI-3 interactive reasoning tasks* (Version 1, arXiv:2512.24156). arXiv. https://doi.org/10.48550/arXiv.2512.24156

Sestak, F., Aichberger, L., Ielanskyi, M., & Klambauer, G. (2026, November 6). *[Public LB: 7] The last dance: Short overview* [Kaggle solution writeup]. ARC Prize 2026 – ARC-AGI-3, Kaggle. Retrieved November 9, 2026, from https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/writeups/public-lb-7-short-overview

Shamim, A. (2026, November 6). *Think about AGI: A Duck based ARC AGI 3 agent and a study of adaptive action verification* [Solution paper, a project file of the Kaggle solution writeup "Think about AGI"]. ARC Prize 2026 – ARC-AGI-3, Kaggle. Retrieved November 9, 2026, from https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/writeups/think-about-agi

Skoutnev, A., Acharya, K., Longhitano, G., Udell, M., Ellis, K., & Drori, I. (2026). *Twin: Playing an unknown game with a test-time digital twin* (Version 1, arXiv:2608.14490). arXiv. https://doi.org/10.48550/arXiv.2608.14490

sonpham-org. (2026). *arc-3: ARC-AGI-3 Duck Harness lab* [GitHub repository README; an instrumented fork of the Tufa Labs harness]. GitHub. Retrieved September 30, 2026, from https://github.com/sonpham-org/arc-3

Spelke, E. S., & Kinzler, K. D. (2007). Core knowledge. *Developmental Science*, *10*(1), 89–96. https://doi.org/10.1111/j.1467-7687.2007.00569.x

Sutton, R. S. (2019, March 13). *The bitter lesson.* Incomplete Ideas. http://www.incompleteideas.net/IncIdeas/BitterLesson.html

Sutton, R. S., Precup, D., & Singh, S. (1999). Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, *112*(1–2), 181–211. https://doi.org/10.1016/S0004-3702(99)00052-1

Taguchi, Y. [ ], shiiin9, Taiiiiiiichi, kTomoya, Sypetkowski, M., & Jaśkowska, N. (2026, November 4). *19th public LB place writeup* [Kaggle solution writeup]. ARC Prize 2026 – ARC-AGI-3, Kaggle. Retrieved November 9, 2026, from https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/writeups/19th-public-lb-place-writeup

Tanaka Ai24, ISAKA Tsuyoshi, re, konbu17, pomcho555, yuto083, TK256, & t fuku. (2026, November 6). *[15th public] Tokens are progress: Serving engineering for an LLM coding agent* [Kaggle solution writeup, with comments]. ARC Prize 2026 – ARC-AGI-3, Kaggle. Retrieved November 9, 2026, from https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/writeups/15th-public-tokens-are-progress-serving-enginee

Team Pandora [Pppycccc & pjh3316864904]. (2026, November 3). *33.39 and a bag of scars: MAXREQ tests, the 0.00 trap, and a REAP autopsy* [Kaggle solution writeup]. ARC Prize 2026 – ARC-AGI-3, Kaggle. Retrieved November 4, 2026, from https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/writeups/33-39-and-a-bag-of-scars-maxreq-tests-the-0-00-t

Tolman, E. C. (1948). Cognitive maps in rats and men. *Psychological Review*, *55*(4), 189–208. https://doi.org/10.1037/h0061626

Tsividis, P. A., Loula, J., Burga, J., Foss, N., Campero, A., Pouncy, T., Gershman, S. J., & Tenenbaum, J. B. (2021). *Human-level reinforcement learning through theory-based modeling, exploration, and planning* (Version 1, arXiv:2107.12544). arXiv. https://doi.org/10.48550/arXiv.2107.12544

Zatravkin, G. (2026a, November 3). *ARC-AGI-3 run curves* [Public rehearsal dashboard; repository commit e5c4bb9]. GitHub Pages. Retrieved November 4, 2026, from `https://gzatravkin.github.io/arc-agi-3-run-curves/`

Zatravkin, G. (2026b, November 2). *Post competition discussion* [Online forum thread, with replies by other participants]. ARC Prize 2026 – ARC-AGI-3, Kaggle. Retrieved November 4, 2026, from `https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/discussion/755202`

Zatravkin, G., Solange, F., _hans, & common-people. (2026, November 4). *6th public (54.55): Let the model tell you where the compute goes* [Kaggle solution writeup]. ARC Prize 2026 – ARC-AGI-3, Kaggle. Retrieved November 4, 2026, from `https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/writeups/6th-public-54-55-let-the-model-tell-you-where-t`

Disclosure

This work was carried out with large language model agents, used deliberately and under a fixed protocol rather than as unreviewed generation. The human author set the research questions, the hypotheses, the working definition and the theoretical contrast of Section 1, and the rules under which the agents worked: a pre-registration with falsification conditions before each submission, a blue team before every red team, a red team run by a model other than the one that built the artifact, a regression test and a mutant for every defect, the full reading of a source before citing it, and, for the final text, block-by-block revision with fidelity to the sources checked before style. The agents carried out the executive work: they implemented the code, prepared the experiments, ran the local tests and benches, read the sources, and drafted the text as a paraphrase of the author's arguments and of the cited works. They also proposed formulations and technical options. Every proposal and every output was adjudicated by the author, who accepted, rejected, or redirected it, block by block, as the text was written and revised. The author ran every Kaggle experiment reported. Models from different providers and of different profiles, Western and Eastern, took the blue and red roles. The author takes responsibility for the content; no language model is an author of this work.